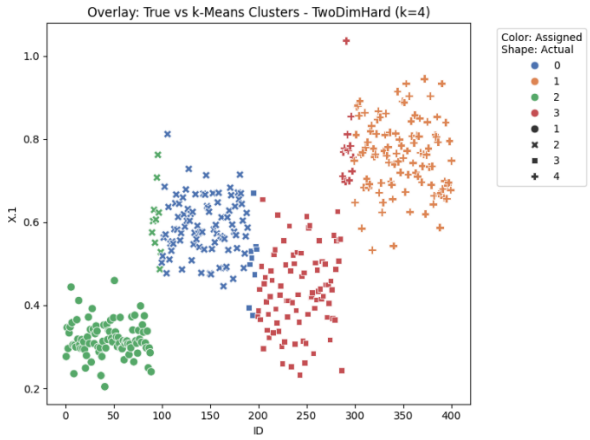
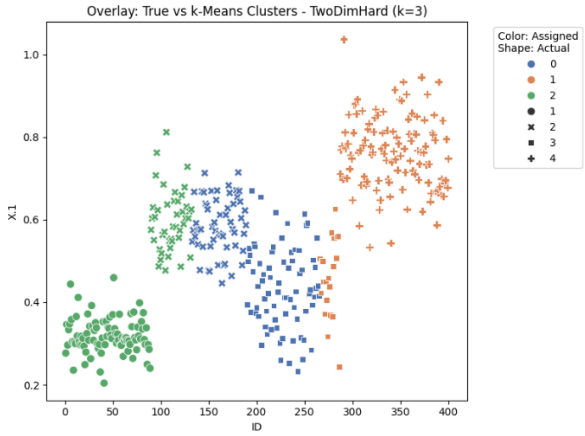
Assignment 4 Write Up

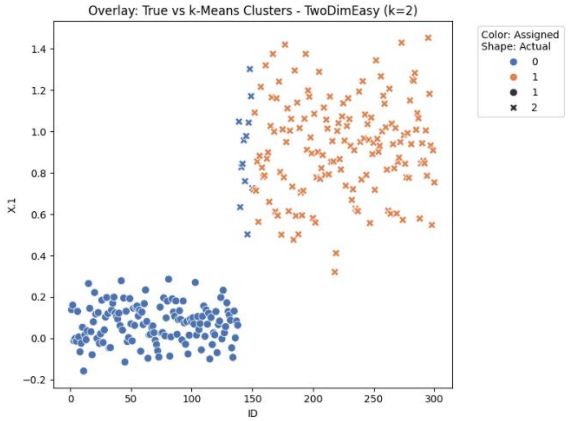
The objective of this assignment is to explore clustering techniques by comparing a custom k-Means implementation with standard off-the-shelf algorithms. The custom k-Means algorithm was implemented manually, using only a select few mathematical operations such as mean, median, standard deviation, etc. It initializes by randomly selecting existing data points to serve as starting centroids, thereby ensuring they fall within the valid data space. During each iteration, the algorithm assigns every data point to the nearest centroid using standard Euclidean distance and subsequently updates each centroid's coordinates based on the mean of its assigned points. This process loops until the centroids converge and no longer shift. To evaluate the algorithm's performance, evaluation metrics such as the Sum of Squared Errors and the Between-cluster Sum of Squares were calculated to measure internal cluster cohesion and external cluster separation, respectively.

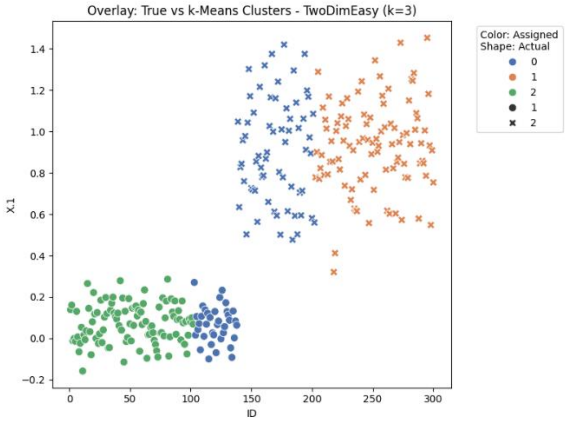
When applied to the TwoDimEasy dataset configured for two clusters, the custom algorithm successfully partitioned the well-separated groups. The predicted overall Sum of Squared Errors of 562506.80 closely mirrored the true cluster metric of 573294.36, and the cross-tabulation matrix showed near-perfect alignment with 138 and 150 correctly assigned points and only 12 misclassifications. This clean separation is visually confirmed by the alignment of true and assigned markers in the attached TwoDimEasy (k=4) plot. However, when the parameters were altered to force three clusters, the algorithm artificially split the natural groupings in half, as illustrated in the TwoDimEasy (k=3) scatterplot. While this mathematically decreased the overall error to 250217.40 due to points being closer to their local centroids, the cross-tabulation revealed that actual cluster 1 was arbitrarily split into 36 and 102 assigned points, signaling an overfit classification. In the TwoDimHard dataset, which features four

slightly overlapping clusters, the algorithm performed well when configured for four clusters, yielding a predicted error of 333459.22 against a true error of 341571.06. It largely maintained accurate groupings but exhibited some small misclassifications along the boundary zones where the clusters intersect, which is visible in the TwoDimHard (k=4) scatterplot. When this dataset was constrained to three clusters, the attached TwoDimHard (k=3) plot shows how the algorithm merged distinct overlapping groups into a single large mass. This caused a significant spike in the overall error to 592729.55 and a drop in the between-cluster sum of squares to 4740599.75, demonstrating how selecting an inappropriate cluster count distorts natural data boundaries.









The Wine Quality dataset introduced a higher-dimensional challenge. Before any clustering occurred, the dependent quality attribute was removed, and the remaining

physicochemical features were scaled to make sure that distance metrics were not heavily skewed by unscaled variances. Experimenting with various cluster counts showed that while the overall error steadily decreased from 14330.12 at k=2 to 7809.30 at k=10, the silhouette scores indicated that the data naturally grouped best at either two (score of 0.214) or six clusters (score of 0.181). Because the actual quality ratings in the dataset span six distinct classes, configuring the model for six clusters provided the most logical basis for external validation. Cross-tabulating these unsupervised assignments against the true quality scores revealed that the physicochemical properties of the wine do not map cleanly to human-rated quality. For example, wines with an actual quality rating of 5 were widely dispersed, placing 289 points into assigned cluster 0, 88 into cluster 1, and 228 into cluster 2. High-quality and low-quality wines were frequently grouped into the same spatial clusters, underscoring the subjective nature of the original quality ratings.

In the second part of the analysis, standard off-the-shelf clustering models, such as Scikit-Learn k-Means, DBSCAN, and Agglomerative Hierarchical, were used on the same datasets to compare their behavior against the custom implementation. The Scikit-Learn k-Means model produced spatial assignments nearly identical to the custom algorithm, matching the custom model's strong performance on the TwoDimEasy dataset with a silhouette score of 0.620. However, it operated with significantly greater efficiency and utilized a superior initialization method that prevented it from converging on an undesired local minimum. DBSCAN, which yielded a lower silhouette score of 0.282 on the easily separated data, proved particularly effective on the overlapping regions of the TwoDimHard dataset. Since it relies on density thresholds rather than rigid geometric distances, it successfully grouped dense core regions while classifying ambiguous boundary points as noise, avoiding the arbitrary boundary assignments

seen in k-Means. Agglomerative Hierarchical clustering matched the k-Means silhouette score of 0.620 on the easy dataset, validating its utility as an alternative approach. Ultimately, building the algorithm from scratch highlighted the strict mathematical mechanics of centroid updating, while the comparative analysis demonstrated that although k-Means is efficient for spherical data, density-based algorithms offer superior handling of overlapping distributions and outliers.